🟢 Git Beginner Interview Mastery — B1 to B4

> **How to use this guide:** Each topic follows a fixed structure — concept → internals → interview answer → deep dive → real-world example → interview questions → gotchas → connections. Use the ⚠ markers to prioritize high-frequency interview traps.

B1. Git Fundamentals

---

1. What Git Is — Distributed vs Centralized VCS

What it is (Simple Terms)

Git is a **version control system** — a tool that tracks changes to files over time, lets multiple people collaborate on the same codebase, and allows you to go back to any previous state of your project.

Why it exists — The Problem it Solves

Before Git, teams used centralized systems like SVN or CVS. Every developer had to connect to a central server to commit code. If the server went down, no one could work. If history was corrupted, it was gone. Git solved this with a **distributed model** — every developer has a complete copy of the entire repository including full history.

Centralized VCS vs Distributed VCS

Feature	Centralized (SVN, CVS)	Distributed (Git)
Repository location	One central server	Every developer has a full copy
Offline work	Not possible	Full capability offline
Single point of failure	Yes	No
Speed	Slow (network calls for everything)	Fast (most ops are local)
Branching	Heavy, slow	Lightweight, instant
History	Lives on server	Lives everywhere

How it Works Internally

Git stores your project as a series of **snapshots**, not diffs. Every time you commit, Git takes a snapshot of all your files and stores a reference to it. If a file hasn't changed, Git doesn't store it again — it just links to the previous identical file. This makes Git fast and space-efficient.

Short Interview Answer

> "Git is a distributed version control system where every developer has a full copy of the repository and its history locally. Unlike centralized systems like SVN, Git doesn't require a network connection for most operations and has no single point of failure. It stores data as snapshots of the filesystem at each commit point."

Deep Dive

Git's distributed nature means operations like `git log`, `git diff`, `git branch`, and `git commit` are all **local operations** with no network latency. The central server (GitHub, GitLab) is just a convention — technically it's just another peer in the distributed network. This is why Git is so fast and why you can work completely offline.

Real-World Production Example

At a company running microservices, each team has their own repo. A developer on an airplane can still commit, branch, merge, and review history without internet. When they land, a single `git push` syncs everything. A centralized VCS would have left them unable to commit a single line of code during the flight.

Common Interview Questions

Q: What is the difference between Git and GitHub?

> Git is the version control tool that runs locally on your machine. GitHub is a cloud-based hosting service for Git repositories. GitHub adds collaboration features like pull requests, issue tracking, and CI/CD integrations on top of Git. You can use Git without GitHub — for example with GitLab, Bitbucket, or a bare repo on your own server.

Q: Why is Git distributed? What advantage does it give?

> Because every developer has the full repo, you can work offline, you have natural backups everywhere, operations are fast since they're local, and there's no single point of failure. It also enables workflows that aren't possible in centralized systems, like local feature branches.

Q: How does Git store data differently from SVN?

> SVN stores data as a list of file-based changes (deltas). Git stores snapshots of the entire project at each commit. Git is more like a mini filesystem with powerful tools built on top of it.

⚠ Gotchas

- Git is **not** a backup system. Just because it's distributed doesn't mean your work is safe — you still need to push to a remote.
- GitHub/GitLab are not Git. Interviewers sometimes conflate the two — be precise.
- Git tracks **content**, not files. Rename a file and Git detects it as a rename if the content is similar enough.

Connections to Other Concepts

- Distributed nature → enables **multi-branch workflows** (B2)
- Local repository → connects to **remote operations** (B3)
- Snapshot model → connects to **Git internals** (Intermediate I1)

2. The Three States — Working Directory, Staging Area, Repository

What it is (Simple Terms)

Every file in a Git project lives in one of three states at any moment:

1. **Working Directory** — where you edit files on disk
2. **Staging Area (Index)** — where you prepare changes before committing
3. **Repository (.git directory)** — where committed snapshots are permanently stored

Why it exists — The Problem it Solves

The staging area is Git's superpower compared to other VCS tools. It lets you **selectively choose what goes into a commit**. You might edit 10 files but only want to commit 3 of them. The staging area is the waiting room between your edits and a permanent commit.

How it Works Internally

```
[Working Directory]  →  git add  →  [Staging Area / Index]  →  git commit
→  [.git Repository]
     (modified)                        (staged)
(committed)
```

The staging area is physically stored in `.git/index` — a binary file that tracks what will go into the next commit. When you run `git add`, Git hashes the file content and writes it into the object store, then updates the index to point to that new blob object.

The Four File States

State	Meaning
Untracked	File exists in working dir, Git has never seen it
Tracked + Unmodified	File matches the last commit
Tracked + Modified	File has changes, not yet staged
Staged	Changes are in the staging area, ready to commit

Short Interview Answer

> "Git has three main states. The working directory is where you edit files. The staging area, also called the index, is where you prepare a set of changes for the next commit. The repository is the .git directory where all committed history lives as immutable objects. The staging area is intentional — it lets you craft precise commits instead of dumping all your changes at once."

Deep Dive

When you run `git add file.txt`:

1. Git reads the file content
2. Computes its SHA-1 hash
3. Writes a **blob object** to `.git/objects/`
4. Updates `.git/index` to reference this blob

When you run `git commit`:

1. Git reads the index
2. Creates a **tree object** from it (directory structure)
3. Creates a **commit object** pointing to that tree and the parent commit
4. Updates the current branch ref (`HEAD`) to point to this new commit

Real-World Production Example

A developer fixes a bug and also refactors some unrelated code in the same session. Using the staging area, they can `git add` only the bug fix files and commit them with message `fix: null pointer exception in auth service`. Then separately stage the refactor and commit it as `refactor: extract auth helper methods`. This produces clean, atomic commits — essential for code review, `git bisect`, and rollback.

Common Interview Questions

Q: What is the difference between the working tree and the staging area?

> The working tree is your local filesystem where you edit files. The staging area (index) is a buffer between the working tree and the repository. Changes in the working tree are not tracked until you explicitly stage them with `git add`.

Q: Can you commit without staging?

> Yes. `git commit -a` stages all **tracked** modified files and commits them in one step. But it skips untracked files. The proper way to be selective is to use `git add` explicitly.

Q: What does `git add .` do?

> It stages all changes in the current directory and subdirectories — new files, modifications, and deletions. It's quick but dangerous if you haven't reviewed what's changed. `git status` first is a good habit.

⚠ Gotchas

- `git add .` stages deletions too — many developers don't realize this.
- A file can be **both staged and modified** at the same time. If you `git add file.txt` and then edit it again, you have a staged version and a newer unstaged version. `git commit` will only commit the staged version.
- `git rm` removes from both working directory AND staging area. `rm file.txt` only removes from working directory — you still need `git add` to stage the deletion.

Connections to Other Concepts

- Staging area → `git add`, `git status`, `git diff` (next section)

- Index is used by Git internally → connects to **Git Internals** (I1)
- `git stash` temporarily shelves working directory changes → (I4)

---

3. Core Commands — `git init`, `git clone`, `git add`, `git commit`, `git status`, `git log`

`git init`

What it does: Creates a new Git repository in the current directory. Creates a `.git/` subdirectory with all the internals — object store, refs, config, hooks directory.

```
mkdir my-project && cd my-project
git init
# Initialized empty Git repository in /my-project/.git/

# Init with a specific branch name (modern practice)
git init --initial-branch=main
# or
git init -b main
```

What's created inside `.git/`:

```
.git/
├── HEAD           # Points to current branch
├── config         # Repo-level config
├── description    # Used by GitWeb
├── hooks/         # Client-side hook scripts
├── info/          # Global gitignore
├── objects/       # All content (blobs, trees, commits, tags)
│   ├── info/
│   └── pack/
└── refs/          # Branch and tag pointers
    ├── heads/     # Local branches
    └── tags/      # Tags
```

`git clone`

What it does: Creates a copy of an existing repository — full history, all branches, all objects.

```
# Clone via HTTPS
git clone https://github.com/org/repo.git

# Clone via SSH (preferred for write access)
git clone git@github.com:org/repo.git

# Clone into a specific directory
```

```
git clone git@github.com:org/repo.git my-folder

# Shallow clone — only last N commits (faster for CI)
git clone --depth=1 git@github.com:org/repo.git

# Clone a specific branch
git clone -b feature/payments git@github.com:org/repo.git

# Clone with submodules
git clone --recurse-submodules git@github.com:org/repo.git
```

What happens internally during clone:

1. Git negotiates with the remote server
2. Downloads all objects (packfile transfer)
3. Unpacks objects into `.git/objects/`
4. Creates remote-tracking refs (`origin/main`, etc.)
5. Checks out the default branch into the working directory

`git add`

What it does: Moves changes from the working directory into the staging area.

```
# Stage a specific file
git add file.txt

# Stage all changes in current directory
git add .

# Stage all changes across the whole repo
git add -A

# Interactively stage parts of a file (hunks)
git add -p file.txt

# Stage all modified tracked files (no new untracked files)
git add -u
```

Difference between `git add .` and `git add -A`:

Command	Stages new files	Stages modifications	Stages deletions
`git add .`	✅ (in current dir)	✅	✅
`git add -A`	✅ (whole repo)	✅	✅
`git add -u`	❌	✅	✅

`git commit`

What it does: Records staged changes as a new commit object in the repository.

```
# Commit with inline message
git commit -m "feat: add user authentication"

# Open editor for longer message
git commit

# Stage all tracked files and commit in one step
git commit -am "fix: resolve null pointer in payment service"

# Amend the last commit (message or content)
git commit --amend -m "fix: resolve null pointer in payment service"

# Commit with detailed message (subject + body)
git commit -m "feat: add OAuth2 login

Implements Google and GitHub OAuth2 providers.
Adds token refresh logic and session management.

Closes #142"
```

What a commit object contains:

```
commit <SHA>
tree <SHA>          ← pointer to the tree (directory snapshot)
parent <SHA>        ← pointer to the previous commit
author Name <email> <timestamp>
committer Name <email> <timestamp>

feat: add user authentication
```

Good commit message conventions (Conventional Commits):

```
feat:     New feature
fix:      Bug fix
docs:     Documentation only
style:    Formatting (no logic change)
refactor: Code change that is not a fix or feature
test:     Adding or updating tests
chore:    Build process or tooling changes
```

`git status`

What it does: Shows the state of the working directory and staging area.

```
git status

# Short format
git status -s
# Output:
# M  file1.txt    ← staged modification (green M)
#  M file2.txt    ← unstaged modification (red M)
# ?? newfile.txt  ← untracked
# A  staged.txt   ← new file staged

# Verbose — shows diff of staged changes
git status -v
```

---

git log

What it does: Shows the commit history of the current branch.

```
# Default log
git log

# One line per commit
git log --oneline

# With graph (essential for visualizing branches)
git log --oneline --graph --all

# Show last N commits
git log -5

# Filter by author
git log --author="John"

# Filter by date
git log --since="2 weeks ago"
git log --after="2024-01-01" --before="2024-06-01"

# Search commit messages
git log --grep="authentication"

# Show commits that changed a specific file
git log -- path/to/file.txt

# Show full diff for each commit
git log -p

# Show stats (files changed, insertions, deletions)
git log --stat

# Pretty format
git log --pretty=format:"%h %an %s %ad" --date=short
```

Understanding `git log --oneline --graph --all` output:

```
* a3f9d2c (HEAD -> main) feat: add payment processing
* b7e1234 fix: resolve login timeout
| * c4d5678 (feature/notifications) feat: add email notifications
|/
* 9a8b765 refactor: extract service layer
* 1f2e3d4 Initial commit
```

Short Interview Answer (Core Commands)

> "The core Git workflow is: `git init` to create a repo or `git clone` to copy one. You edit files in the working directory, `git add` to stage the changes you want, and `git commit` to permanently record them. `git status` tells you where everything stands, and `git log` shows you the commit history. These six commands are the backbone of every Git workflow."

Common Interview Questions

Q: What is the difference between `git add .` and `git add -A`?

> `git add .` stages everything in the current directory and below. `git add -A` stages everything across the entire repository regardless of where you currently are in the directory structure.

Q: What happens if you run `git commit` with nothing staged?

> Git will say "nothing to commit, working tree clean" and abort. Unless you use `--allow-empty`, which creates an empty commit — sometimes used for triggering CI pipelines.

Q: How do you write a good commit message?

> A good commit message has a short subject line (50 chars or less) in the imperative mood — "Add feature" not "Added feature". Optionally followed by a blank line and a body explaining the why, not the what. Conventional Commits format (`feat:`, `fix:`, `docs:` etc.) is widely adopted in DevOps pipelines for automated changelog generation.

⚠ Gotchas

- `git commit -am` only stages **tracked** files. New untracked files still need `git add`.
- `git log` only shows history for the **current branch** by default. Use `--all` to see all branches.
- Amending a commit with `--amend` rewrites history. Never amend commits that have already been pushed to a shared branch.
- SHA hashes shown in `git log --oneline` are abbreviated. The full SHA is 40 characters.

---

4. `.gitignore` — Patterns and Purpose

What it is (Simple Terms)

A `.gitignore` file tells Git which files and directories to **ignore** — to never track, never stage, and never commit. These are typically build artifacts, dependencies, secrets, and editor files.

Why it exists

Without `.gitignore`, running `git add .` would stage compiled binaries, `node_modules` (potentially millions of files), `.env` files with secrets, IDE config files, and OS junk files like `.DS_Store`. The repository would become bloated, insecure, and noisy.

Pattern Syntax

```
# Comments start with #

# Ignore a specific file
.env
secrets.json

# Ignore a file type anywhere in the repo
*.log
*.pyc
*.class

# Ignore a specific directory
node_modules/
dist/
build/
.terraform/

# Ignore files only at the root level (leading slash)
/config.local.json

# Ignore a file in a specific subdirectory
src/generated/

# Negate a pattern — track even if a parent rule ignores it
!important.log

# Ignore all .txt files in any 'docs' directory
docs/**/*.txt

# Double asterisk = match across directories
**/temp/
```

Common `.gitignore` Entries by Stack

```
# Node.js
node_modules/
npm-debug.log
.env
```

```
.env.local
dist/

# Python
__pycache__/
*.pyc
*.pyo
.venv/
*.egg-info/
.pytest_cache/

# Java / Maven
target/
*.class
*.jar

# Terraform
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars    # Be careful — sometimes these should be tracked

# Docker
.dockerignore  # Separate file for Docker build context

# IDEs
.idea/
.vscode/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db
```

Global `.gitignore`

For patterns that should apply to ALL your repos (like IDE files), use a global ignore file:

```
git config --global core.excludesfile ~/.gitignore_global
echo ".DS_Store" >> ~/.gitignore_global
echo ".idea/" >> ~/.gitignore_global
```

`.gitignore` vs `.git/info/exclude`

- `.gitignore` is tracked by Git and shared with the team
- `.git/info/exclude` is local only — for your personal ignores that shouldn't be shared

Short Interview Answer

> "`.gitignore` tells Git which files to never track. The patterns use glob syntax — you can ignore specific files, entire directories, file extensions, or use negation patterns. Common ignores are build artifacts, dependency directories like `node_modules`, secrets like `.env` files, and IDE configuration. I always generate one from `gitignore.io` as a starting point and add project-specific patterns."

Common Interview Questions

Q: What happens if you commit a file and then add it to `.gitignore`?

> Nothing. `.gitignore` only affects **untracked** files. If a file is already committed and tracked, `.gitignore` has no effect on it. You need to run `git rm --cached filename` to untrack it while keeping the local copy, then commit the removal.

Q: How do you stop tracking a file that's already committed?

```
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "chore: untrack .env file"
```

Q: What's the difference between `.gitignore` and `.dockerignore`?

> `.gitignore` controls what Git tracks. `.dockerignore` controls what Docker includes in the build context when building images. They often overlap but serve different tools.

⚠ Gotchas

- `.gitignore` doesn't apply retroactively. Files already tracked stay tracked.
- Directories must end with `/` in the pattern to specifically match directories. `build` matches both a file and directory named `build`. `build/` only matches directories.
- Secrets committed even once are in Git history forever — even if you delete and add to `.gitignore`. You need `git filter-repo` or BFG to truly purge them.
- Negation patterns (`!`) can't override rules from a parent directory's `.gitignore`.

---

5. Local vs Remote Repositories

What it is (Simple Terms)

- **Local repository** — the `.git` directory on your machine. All commits, branches, tags, and history live here.
- **Remote repository** — a copy of the repo hosted on a server (GitHub, GitLab, Bitbucket, or your own server). Acts as the shared source of truth for a team.

How it Works

```
[Developer A Laptop]          [GitHub / GitLab]          [Developer B
Laptop]
```

```
    Local Repo          ←→      Remote Repo (origin)  ←→     Local Repo
    .git/                       main, develop, etc.          .git/
```

Git stores remote information in `.git/config`:

```
[remote "origin"]
    url = git@github.com:myorg/myrepo.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
```

Remote-Tracking Branches

When you clone or fetch, Git creates **remote-tracking branches** — read-only local pointers to the state of branches on the remote at the time of last fetch. They are stored as `refs/remotes/origin/main`, `refs/remotes/origin/develop`, etc.

```
# See all branches including remote-tracking
git branch -a

# Output:
# * main
#   feature/payments
#   remotes/origin/HEAD -> origin/main
#   remotes/origin/main
#   remotes/origin/develop
```

Short Interview Answer

> "The local repository is the full `.git` directory on your machine — it contains all commits, branches, and history and you can work with it completely offline. The remote repository is a shared copy on a server like GitHub. Remote-tracking branches like `origin/main` are local read-only snapshots of what the remote looked like the last time you fetched."

⚠ Gotchas

- You can have **multiple remotes** — `origin`, `upstream`, `backup`. This is common in open-source where `origin` is your fork and `upstream` is the original repo.
- Remote-tracking branches are **not automatically updated** when the remote changes. You need `git fetch` to update them.

---

6. How Commits Are Identified — SHA Hashes

What it is (Simple Terms)

Every commit, file, directory, and tag in Git is identified by a **SHA-1 hash** — a 40-character hexadecimal string like `a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d`. This hash is computed from the **content** of the object, making it a content-addressed storage system.

Why SHA Hashes Matter

```
# Full SHA
git log --format="%H"
# a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d

# Abbreviated SHA (7 chars, shown in --oneline)
git log --oneline
# a3f9d2c feat: add authentication

# Git only needs enough chars to be unambiguous
git show a3f9d2c       # works
git show a3f9          # works if unique in repo
git checkout a3f9d2c   # checkout a specific commit
```

Why SHA-1 for Integrity

The SHA-1 hash is computed from the commit's content (message, author, timestamp, tree pointer, parent pointer). This means:

- If **any byte** changes in any commit, its hash changes
- If a commit's hash matches, its content is guaranteed to be unchanged
- This provides **cryptographic data integrity** across all clones
- Git is slowly migrating to SHA-256 for future security

Short Interview Answer

> "Every Git object is identified by a SHA-1 hash computed from its content. This means Git is a content-addressed store — if the content changes, the hash changes. Two commits with the same hash are guaranteed to be identical. This is how Git ensures data integrity across distributed clones. In practice, you only need the first 7 or so characters as long as they're unique in the repo."

⚠ Gotchas

- SHA-1 has known theoretical collision vulnerabilities. GitHub and GitLab have protections against SHA-1 collision attacks. Git is migrating to SHA-256.
- When you **amend or rebase**, commits get **new SHAs** because their content changes. This is why rebasing published commits causes problems — collaborators still reference the old SHAs.

---

7. First-Time Git Config — `git config --global`

What it is

Git config stores settings at three levels: **system** (all users), **global** (current user), and **local** (current repo). These control your identity, editor, default behaviors, and much more.

Essential First-Time Setup

```
# Set your identity (required — embedded in every commit)
git config --global user.name "Jane Smith"
git config --global user.email "jane@company.com"

# Set default editor (for commit messages, rebases, etc.)
git config --global core.editor "vim"
git config --global core.editor "code --wait"    # VS Code
git config --global core.editor "nano"

# Set default branch name for new repos
git config --global init.defaultBranch main

# Enable colored output
git config --global color.ui auto

# Set line ending handling
# macOS / Linux:
git config --global core.autocrlf input
# Windows:
git config --global core.autocrlf true

# Set default pull behavior (rebase instead of merge)
git config --global pull.rebase true

# Cache credentials (HTTPS)
git config --global credential.helper cache
git config --global credential.helper 'cache --timeout=3600'

# Set up useful aliases
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.unstage "reset HEAD --"
```

Config File Locations

```
/etc/gitconfig          → system level (all users on machine)
~/.gitconfig            → global level (current user, all repos)
<repo>/.git/config      → local level (current repo only)
```

Local overrides global, global overrides system.

```
# View all config and where each setting comes from
git config --list --show-origin

# View a specific setting
git config user.email

# Set a repo-level setting (overrides global)
git config user.email "work@company.com"   # inside a work repo
```

Short Interview Answer

> "Git config stores settings at three levels — system, global, and local. For first-time setup, you must set `user.name` and `user.email` as these are baked into every commit you make. I also set `init.defaultBranch main`, configure my editor, and set up aliases for common commands. The `--global` flag writes to `~/.gitconfig` and applies to all repos. Local config in `.git/config` overrides global, which is useful when you have separate work and personal configs."

⚠ Gotchas

- If you forget to set `user.email` at a job with a different email, you'll make commits with your personal email embedded in the company's repo history.
- Setting `core.autocrlf=true` on Linux can corrupt binary files committed on Windows.
- `git config --global --unset <key>` removes a setting.

B2. Branching and Merging

---

1. What Branches Are — Lightweight Pointers to Commits

What it is (Simple Terms)

A branch in Git is simply a **movable pointer to a commit**. That's it. It's just a file containing a 40-character SHA-1 hash. When you create a branch, Git creates a new file in `.git/refs/heads/`. When you commit, the branch pointer moves forward automatically.

Why This Matters

In older VCS tools, branches were expensive copies of the entire codebase. In Git, a branch is a 41-byte file. Creating a branch is **instant and free**. This is why Git encourages heavy use of branches — for every feature, every bug fix, every experiment.

How it Works Internally

```
.git/refs/heads/main     → contains:
a3f9d2c4e7b18f6d9c0e5a2b3d4f7e8c9a0b1c2d
```

```
.git/refs/heads/feature  → contains:
b7e12340f5a8b9c6d1e2f3a4b5c6d7e8f9a0b1c2
```

```
        HEAD
         │
         ▼
        main ──────────────► Commit C3 → Commit C2 → Commit C1
       feature ─────────────► Commit F2 → Commit F1 → Commit C1 (shared
ancestor)
```

HEAD — Where You Are Right Now

`HEAD` is a special pointer that tells Git which branch (or commit) you currently have checked out. It lives in `.git/HEAD`.

```
cat .git/HEAD
# ref: refs/heads/main      ← attached to main branch
```

When you commit, Git:

1. Creates the new commit object
2. Moves the branch pointer (e.g., `main`) to the new commit
3. `HEAD` stays pointing to the branch, so it effectively moves too

Short Interview Answer

> "A branch in Git is literally just a lightweight pointer — a file containing a SHA-1 hash pointing to a commit. When you commit, the pointer moves forward automatically. This is fundamentally different from older tools where branches were heavyweight copies. In Git, creating a branch takes microseconds and uses virtually no disk space."

---

2. Creating, Switching, Renaming, Deleting Branches

```
# Create a branch (doesn't switch to it)
git branch feature/login

# Create and switch in one step (traditional)
git checkout -b feature/login

# Create and switch (modern, Git 2.23+)
git switch -c feature/login

# Create a branch from a specific commit or tag
git checkout -b hotfix/payment-bug v2.1.0
git checkout -b feature/new-ui abc1234
```

```
# Switch between branches
git checkout main
git switch main          # modern syntax

# List all local branches
git branch

# List with last commit info
git branch -v

# List all (local + remote)
git branch -a

# Rename a branch
git branch -m old-name new-name

# Rename current branch
git branch -m new-name

# Delete a merged branch (safe)
git branch -d feature/login

# Force delete (even if unmerged) ⚠ Data loss risk
git branch -D feature/experiment

# Delete a remote branch
git push origin --delete feature/login
# or
git push origin :feature/login
```

---

3. Merging — Fast-Forward vs Three-Way Merge

Fast-Forward Merge

Occurs when the branch you're merging **has no diverged history** — the target branch is a direct ancestor.

```
Before:
  main ──────► C1 → C2
                         \
  feature ─────────────────► F1 → F2

After fast-forward merge of feature into main:
  main ──────────────────────────────► F2 (pointer just moved forward)
  feature ────────────────────────────► F2 (same commit)

No merge commit created.
```

```
git checkout main
git merge feature/login
# Fast-forward merge, no new commit
```

Three-Way Merge

Occurs when both branches have **diverged** — each has commits the other doesn't have.

```
Before:
  main:    C1 → C2 → C3
                \
  feature:       F1 → F2

After three-way merge:
  main:    C1 → C2 → C3 → M (merge commit)
                \        /
  feature:       F1 → F2
```

Git uses **three commits** to create the merge:

1. The tip of the current branch (C3)
2. The tip of the branch being merged (F2)
3. The **common ancestor** (C2) — used to determine what changed in each

```
git checkout main
git merge feature/login
# Creates a merge commit with two parents
```

Forcing a Merge Commit (No Fast-Forward)

```
# Always create a merge commit even if fast-forward is possible
git merge --no-ff feature/login
# Preserves the fact that a feature branch existed in history
```

Comparison

	Fast-Forward	**Three-Way Merge**
When	No diverged history	Both have unique commits
Merge commit	No	Yes
History	Linear	Non-linear (branching visible)
`--no-ff` override	Yes	N/A

Short Interview Answer

> "Git has two main merge types. A fast-forward merge happens when the base branch has no new commits since the feature branch was created — Git just moves the pointer forward, no merge commit is created. A three-way merge happens when both branches have diverged — Git finds the common ancestor, compares both tips against it, and creates a merge commit with two parents. `--no-ff` forces a merge commit even when fast-forward is possible, which preserves branch history in the log."

---

4. Merge Conflicts — What Causes Them and How to Resolve

What Causes Conflicts

A conflict occurs when **both branches modified the same lines** of the same file, and Git cannot automatically determine which change to keep. Git can auto-merge changes to different lines or different files — it only fails when the same region was edited in incompatible ways.

What a Conflict Looks Like

```
<<<<<<< HEAD
const timeout = 5000;
=======
const timeout = 10000;
>>>>>>> feature/performance-tuning
```

- `<<<<<<< HEAD` — your current branch's version
- `=======` — separator
- `>>>>>>> feature/...` — the incoming branch's version

How to Resolve Conflicts

```
# Step 1: See which files have conflicts
git status
# both modified: src/config.js

# Step 2: Open the file and resolve manually (or use a tool)
# Edit the file to the correct final state, removing markers

# Step 3: Stage the resolved file
git add src/config.js

# Step 4: Complete the merge
git commit
# Git pre-fills the merge commit message

# --- Using a merge tool ---
git mergetool          # Opens configured diff/merge tool
```

```
git mergetool --tool=vimdiff
git mergetool --tool=vscode
```

Setting up VS Code as merge tool:

```
git config --global merge.tool vscode
git config --global mergetool.vscode.cmd 'code --wait $MERGED'
```

Aborting a Merge

```
# If you want to start over and undo the merge
git merge --abort
```

⚠ Gotchas

- Conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`) must be **completely removed** from the file before staging. Staging a file with conflict markers still in it is a common mistake.
- Binary files (images, PDFs) cannot be auto-merged. Git marks them as conflicted and you must choose one version: `git checkout --ours image.png` or `git checkout --theirs image.png`.
- A merge can have **multiple conflicted files**. Resolve and `git add` each one individually, then one `git commit` at the end.

---

5. HEAD and Detached HEAD State

Normal State (Attached HEAD)

```
HEAD → refs/heads/main → Commit SHA
```

HEAD points to a branch. When you commit, the branch moves forward and HEAD moves with it.

Detached HEAD State

```
HEAD → Commit SHA (directly, no branch)
```

Occurs when you check out a specific commit SHA, a tag, or a remote-tracking branch directly.

```
# These all cause detached HEAD:
git checkout abc1234           # checking out a commit SHA
```

```
git checkout v2.1.0            # checking out a tag
git checkout origin/main       # checking out a remote-tracking branch
```

```
$ git checkout abc1234
Note: switching to 'abc1234'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.
```

Why It's Dangerous

In detached HEAD, if you make commits and then switch to a branch, **those commits become unreferenced** and will eventually be garbage collected. They appear in `git reflog` for a while but are not protected.

How to Recover

```
# Option 1: Create a branch from current detached HEAD position
git checkout -b rescue-branch

# Option 2: If you already switched away, find the SHA in reflog
git reflog
# HEAD@{1}: abc1234
git checkout -b rescue-branch abc1234
```

Short Interview Answer

> "HEAD is Git's pointer to your current position. In normal state, HEAD points to a branch, and committing moves that branch forward. Detached HEAD happens when HEAD points directly to a commit SHA instead of a branch — typically from checking out a specific commit or tag. In detached HEAD, commits you make are orphaned when you switch away because no branch tracks them. To preserve work from detached HEAD, create a branch immediately."

---

6. `git diff` — Comparing Working Tree, Staging, Commits

```
# Diff between working directory and staging area (unstaged changes)
git diff

# Diff between staging area and last commit (staged changes)
git diff --staged
git diff --cached     # same thing

# Diff between two commits
```

```
git diff abc1234 def5678

# Diff between two branches
git diff main feature/login

# Diff a specific file between branches
git diff main feature/login -- src/auth.js

# Diff working directory against a specific commit
git diff HEAD~3

# Show only file names that changed (not the actual diff)
git diff --name-only main feature/login

# Show summary (files changed, insertions, deletions count)
git diff --stat main feature/login

# Word-level diff (useful for prose)
git diff --word-diff
```

Reading Diff Output

```
diff --git a/src/auth.js b/src/auth.js
index a1b2c3d..e4f5g6h 100644
--- a/src/auth.js          ← old version (a/)
+++ b/src/auth.js          ← new version (b/)
@@ -10,7 +10,8 @@         ← hunk header: old start,count new start,count
 function authenticate(user) {
-  const timeout = 5000;   ← removed line (red)
+  const timeout = 10000;  ← added line (green)
+  const retries = 3;      ← added line (green)
   return fetch(url);
 }
```

⚠ Gotchas

- `git diff` with no arguments shows **unstaged** changes only. Many people expect it to show all changes but staged changes are invisible to it.
- `git diff --staged` is needed to see what will actually go into the next commit.
- In CI pipelines, `git diff origin/main...HEAD --name-only` is commonly used to find which files changed in a PR for targeted testing.

B3. Remote Operations

---

1. `git remote` — Adding and Managing Remotes

What it is

A remote is a named reference to another Git repository — usually on a server like GitHub. The name `origin` is just a convention for the first remote.

```
# View remotes (name only)
git remote

# View remotes with URLs
git remote -v
# origin  git@github.com:myorg/myrepo.git (fetch)
# origin  git@github.com:myorg/myrepo.git (push)

# Add a remote
git remote add origin git@github.com:myorg/myrepo.git

# Add a second remote (common in open source — origin=your fork,
upstream=original)
git remote add upstream git@github.com:originalorg/myrepo.git

# Change a remote URL
git remote set-url origin git@github.com:myorg/new-repo-name.git

# Rename a remote
git remote rename origin old-origin

# Remove a remote
git remote remove upstream

# Show detailed info about a remote (fetch/push URLs, tracking branches)
git remote show origin
```

⚠ 2. `git fetch` vs `git pull`

This is one of the **most commonly asked beginner-to-intermediate Git interview questions**.

`git fetch`

Downloads changes from the remote into **remote-tracking branches** but does **NOT** modify your working directory or local branches. It is always safe.

```
# Fetch all remotes
git fetch

# Fetch specific remote
git fetch origin

# Fetch specific branch
git fetch origin main
```

```
# Fetch all branches and tags
git fetch --all --tags
```

After `git fetch origin`:

```
remote-tracking branch 'origin/main' is updated
your local 'main' branch is UNCHANGED
```

`git pull`

`git pull` is a **two-step operation**: `git fetch` followed by either `git merge` or `git rebase` into your current branch.

```
git pull                      # fetch + merge (default)
git pull --rebase             # fetch + rebase (cleaner history)
git pull origin main          # fetch and merge origin/main into current
branch
```

The Difference — Visualized

```
State: You and the remote have both made new commits

After git fetch:
  local main:    A → B → C (yours)
  origin/main:   A → B → D → E (remote, now visible locally)
  Working dir:   unchanged, still at C

After git pull (= fetch + merge):
  local main:    A → B → C → M (merge commit)
                             ↗
                       D → E
  Working dir:   updated to M
```

Short Interview Answer

> "`git fetch` downloads changes from the remote and updates remote-tracking branches like `origin/main`, but it never touches your local branches or working directory — it's always safe. `git pull` is fetch plus merge (or rebase), so it also updates your current branch. I prefer `git fetch` followed by manual inspection and then merge or rebase, because it gives me control. Using `git pull` blindly can create unexpected merge commits."

⚠ Gotchas

- `git pull` on a team branch can create noisy merge commits. `git pull --rebase` keeps history linear.
- `git fetch` does NOT fetch tags by default in all configurations. Use `git fetch --tags`.
- In CI pipelines, you often see `git fetch origin` followed by `git checkout origin/main` — fetching first then checking out the specific remote state.

---

3. `git push` and Tracking Branches

```
# Push current branch to its upstream
git push

# Push a specific branch to remote
git push origin feature/login

# Push and set upstream tracking in one step (-u flag)
git push -u origin feature/login

# Push all branches
git push --all

# Push tags
git push --tags
git push origin v2.1.0      # push specific tag

# Force push (⚠ DANGER — overwrites remote history)
git push --force
git push --force-with-lease  # safer — fails if remote has new commits

# Delete a remote branch
git push origin --delete feature/old-branch
```

Tracking Branches

A tracking branch is a local branch that has a defined relationship with a remote branch. This relationship tells Git:

- Where to push to by default
- Where to pull/fetch from
- How far ahead/behind you are

```
# See tracking relationships
git branch -vv
# * main           a3f9d2c [origin/main: ahead 2] feat: add auth
#   feature/login  b7e1234 [origin/feature/login] initial

# Set tracking for existing branch
git branch --set-upstream-to=origin/main main
```

4. Upstream Branches and the `-u` Flag

```
# -u (--set-upstream) sets the tracking relationship when pushing
git push -u origin feature/login
# Sets origin/feature/login as the upstream for local feature/login
# After this, plain 'git push' and 'git pull' work without arguments

# Check tracking info
git status
# On branch feature/login
# Your branch is ahead of 'origin/feature/login' by 1 commit.
```

5. Shallow Clones (`--depth`)

What it is

A shallow clone downloads only the last N commits rather than the full history.

```
# Clone with last 1 commit only
git clone --depth=1 git@github.com:org/repo.git

# Clone with last 10 commits
git clone --depth=10 git@github.com:org/repo.git

# Fetch with depth limit (useful in CI)
git fetch --depth=1 origin main

# Convert shallow clone to full clone
git fetch --unshallow
```

Why it's Used in CI/CD

For large repos, a full clone can take minutes. In CI, you often only need the latest commit. `--depth=1` can reduce clone time from minutes to seconds.

⚠ Gotchas with Shallow Clones

- `git log` only shows the cloned history depth — you can't see older commits
- `git describe` fails because it can't find the nearest tag
- `git blame` may not show complete history
- Some merge operations fail due to missing common ancestors
- Always use `--unshallow` if you need full history in CI (e.g., for changelog generation)

B4. Basic Collaboration

---

1. Forking Workflow

What it is

The fork workflow is the standard for **open-source contribution** and is also used in enterprises to isolate teams. Instead of cloning the central repo and pushing directly, you:

1. **Fork** — create your own server-side copy of the repo
2. **Clone** your fork locally
3. **Make changes** in feature branches
4. **Push** to your fork
5. **Open a PR** from your fork to the original (upstream) repo

```
# After forking on GitHub:
git clone git@github.com:YOUR_USERNAME/repo.git
cd repo

# Add the original as upstream
git remote add upstream git@github.com:ORIGINAL_ORG/repo.git

# Verify
git remote -v
# origin   git@github.com:YOUR_USERNAME/repo.git (fetch)
# origin   git@github.com:YOUR_USERNAME/repo.git (push)
# upstream git@github.com:ORIGINAL_ORG/repo.git (fetch)
# upstream git@github.com:ORIGINAL_ORG/repo.git (push)

# Create a feature branch
git checkout -b feat/improve-docs

# ... make changes ...

git push -u origin feat/improve-docs
# Now open a PR on GitHub from your fork to upstream
```

Keeping Your Fork in Sync

```
# Fetch latest from upstream
git fetch upstream

# Merge or rebase your main with upstream's main
git checkout main
git merge upstream/main
# or
git rebase upstream/main

# Push updated main to your fork
git push origin main
```

---

2. Pull Requests and Merge Requests

What they are

A **Pull Request (PR)** (GitHub/Bitbucket) or **Merge Request (MR)** (GitLab) is a proposal to merge one branch into another. It creates a conversation space for code review before changes reach the main branch.

What a PR contains:

- Diff of all changes
- Commit history
- Comment threads on specific lines
- CI/CD status checks
- Review approvals/rejections
- Merge strategy options (merge commit, squash, rebase)

PR Workflow in a Team

```
# Developer creates feature branch
git checkout -b feat/add-oauth

# ... work ...
git push -u origin feat/add-oauth

# Open PR on GitHub UI:
# base: main ← compare: feat/add-oauth

# CI runs automatically (lint, test, build)
# Reviewers comment, request changes
# Developer addresses feedback:
git add .
git commit -m "fix: address review comments on OAuth scope"
git push origin feat/add-oauth   # PR updates automatically

# After approval, merge on GitHub
```

Merge Strategies in PRs

Strategy	What it does	When to use
Merge commit	Creates a merge commit, preserves full branch history	Default for most teams
Squash and merge	Squashes all PR commits into one	Clean history, many small/WIP commits
Rebase and merge	Replays commits on main, linear history	Teams preferring linear history

3. Code Review Process Basics

What reviewers look for:

- **Correctness** — does the code do what the PR says?
- **Security** — any vulnerabilities, exposed secrets?
- **Test coverage** — are changes tested?
- **Design** — is the approach sound?
- **Style** — consistent with codebase conventions?
- **Documentation** — is anything unclear?

Review comment types (GitHub):

- **Comment** — general discussion, no blocking
- **Approve** — LGTM, ready to merge
- **Request changes** — blocking feedback, must be addressed

```
# Developer responds to review feedback:
# 1. Make changes
# 2. Push to the same branch — PR updates automatically
# 3. Resolve conversations
# 4. Re-request review

# Good practice: responding to each comment
# "Done — refactored into a separate helper function"
# "Agreed — added test for null case"
```

4. Branch Protection Rules

What they are

Branch protection rules (GitHub, GitLab, Bitbucket) prevent direct pushes to important branches and enforce review/CI requirements.

Common Protection Rules for `main`:

```
✅ Require pull request before merging
✅ Require at least 1 approval
✅ Dismiss stale pull request approvals when new commits are pushed
✅ Require status checks to pass before merging
   - CI/CD pipeline must be green
   - Test coverage threshold must pass
✅ Require branches to be up to date before merging
✅ Restrict who can push to this branch
❌ Allow force pushes → DISABLED
❌ Allow deletions → DISABLED
```

Why This Matters for SRE/DevOps

Branch protection is a **code quality and deployment safety gate**. Without it:

- Broken code can be pushed directly to `main` and trigger a production deployment
- A single developer can bypass all review processes
- Force pushes can destroy shared commit history

⚠ Interview Insight

Senior engineers are expected to know that **branch protection rules are a pillar of a safe CI/CD pipeline**, not just a collaboration nicety. Connecting this to "shift-left security" and "preventing accidental deploys" scores points.

---

5. Tagging — Lightweight vs Annotated Tags

What Tags Are

Tags are pointers to specific commits, typically used to mark **release versions**. Unlike branches, tags don't move.

```
# Lightweight tag — just a pointer to a commit (like a branch that doesn't
move)
git tag v2.1.0

# Annotated tag — a full Git object with message, author, date, GPG
signature
git tag -a v2.1.0 -m "Release version 2.1.0"

# Tag a specific commit (not the latest)
git tag -a v1.9.0 abc1234 -m "Retroactive tag for v1.9.0"

# List all tags
git tag
git tag -l "v2.*"    # filter by pattern

# Show tag details
git show v2.1.0

# Push tags to remote (tags are NOT pushed with git push by default)
git push origin v2.1.0       # push one tag
git push --tags               # push all tags

# Delete a tag
git tag -d v2.1.0             # local
git push origin --delete v2.1.0   # remote

# Checkout at a tagged version (causes detached HEAD)
git checkout v2.1.0
```

Lightweight vs Annotated Comparison

	Lightweight	Annotated
Storage	Just a ref file	Full Git object
Message	No	Yes
Author/Date	No	Yes
GPG signing	No	Yes
Shown by `git describe`	Yes (with `--tags`)	Yes (default)
Recommended for releases	No	**Yes**

Short Interview Answer

> "Tags mark specific points in history, typically releases. Lightweight tags are just a pointer — like a branch that never moves. Annotated tags are full Git objects with a tagger, date, and message, and can be GPG-signed. For official releases I always use annotated tags because they carry metadata and are returned by `git describe` by default. Tags must be pushed to remote explicitly — `git push --tags` — because `git push` doesn't include them by default."

⚠ Gotchas

- Tags are **not pushed** by default with `git push`. This is a very common gotcha — CI pipelines that depend on tags to compute versions often fail because the tags weren't fetched.
- Deleting a published tag is dangerous if others have referenced it in their pipelines or package managers.

---

6. `git show` and `git log --oneline --graph`

```
# git show — inspect any Git object

# Show a commit (diff + metadata)
git show abc1234

# Show a specific file at a specific commit
git show abc1234:src/auth.js

# Show a tag
git show v2.1.0

# Show the most recent commit
git show HEAD

# Show parent of current commit
git show HEAD~1
git show HEAD^    # same thing
```

```
# Show grandparent
git show HEAD~2

# --- git log visual graph ---

# Most useful command for understanding branch structure
git log --oneline --graph --all --decorate

# Example output:
# * a3f9d2c (HEAD -> main, origin/main) feat: add OAuth login
# * b7e1234 fix: resolve session timeout
# | * c4d5678 (feature/notifications) feat: push notifications
# | * d5e6789 feat: notification service setup
# |/
# * 9a8b765 refactor: extract auth module
# * 1f2e3d4 Initial commit
```

Reading the Graph

- `*` = a commit
- `|` = branch line
- `/` or `\` = branch diverge/converge
- `(HEAD -> main)` = where HEAD and main point
- `(origin/main)` = where remote-tracking branch is

🔗 How Everything in B1–B4 Connects

```
git config (B1) → sets your identity embedded in every commit
     ↓
git init / clone (B1) → creates local repo or copies remote
     ↓
Three states (B1) → Working Dir → Staging Area → Repository
     ↓
.gitignore (B1) → controls what enters the staging area
     ↓
git add/commit (B1) → moves changes through the three states
     ↓
Branches (B2) → parallel lines of development, lightweight pointers
     ↓
Merging (B2) → brings branches back together
     ↓
git remote / fetch / pull / push (B3) → syncing with team via remote
     ↓
Forking / PRs / Branch Protection (B4) → team collaboration and safety
gates
     ↓
Tags (B4) → marking releases → triggers CI/CD deployments
```

```
      ↓
Connects to → Intermediate topics: rebase, internals, hooks, branching
strategies
```

📋 Quick Reference — Most Frequently Asked Beginner Questions

Question	Answer in One Line
`git fetch` vs `git pull`	fetch = download only; pull = fetch + merge
Fast-forward vs three-way merge	FF = linear history, no merge commit; 3-way = diverged, creates merge commit
`git add .` vs `git add -A`	`.` = current dir; `-A` = whole repo
Lightweight vs annotated tag	Lightweight = pointer only; annotated = full object with message
Detached HEAD	HEAD points to commit SHA directly, not a branch
How to untrack a committed file	`git rm --cached <file>` then add to `.gitignore`
How to undo last commit (keep changes)	`git reset --soft HEAD~1`
How to see what will be committed	`git diff --staged`
How to see branch tracking	`git branch -vv`
How to delete a remote branch	`git push origin --delete branchname`

Next: B2 Deep Dive or move to Intermediate I1 (Git Internals — blob, tree, commit, DAG)